

3-Level Geometric Modeler: CSG, Subdivision Trees and Boundary Representation

KAI CAO, ZHONGDI LUO

New York University
kcao@cs.nyu.edu zl562@nyu.edu

Abstract

The problem of computing geometry modeling in $\mathbb{R}^3$ is fundamental to computer graphics, path finding, motion planning and numerous branches of computer science. Given a polyhedron P defined by constructive solid geometry (CSG) model, we raise a new idea to present the boundary features (vertices, edges and faces) of P using subdivision method; it has the following steps: 1) define P in CSG model, 2) subdivide a given box B_0 for P and construct the adequate subdivision tree T , 3) represent the boundary features based on the subdivision tree T . We could also compute the boolean operations of 2 subdivision trees. Since the great generality and scalability of subdivision method, this idea can also be expanded to support more complex shape with higher order surfaces in $\mathbb{R}^3$.

I. INTRODUCTION

In this paper, we focus on the geometric modeling of solids (especially polyhedra) in $\mathbb{R}^3$. The traditional definition of a polyhedron in $\mathbb{R}^3$ is the boolean union of half spaces (CSG). The faces of a polyhedron P are the plains which bounds the half spaces. Meanwhile the edges are the intersection of two unparallel plains and the vertices are the intersection of three. Faces, edges and vertices constitute the features of a polyhedron. By CSG definition, these features are not obvious. To present them more efficiently and make them more operable, we keep subdividing a box B_0 containing P until

the boundary features are kept in certain sub-boxes. Then we can compute the features by manipulating the subdivision tree instead.

II. REGULARIZED SET

A set $S \subset \mathbb{R}^3$ is a **regularized set** if $S = \overline{\text{int}(S)}$.

A **solid** is defined as a regularized subset of $\mathbb{R}^3$. Obviously a polyhedron P is a finite solid.

Suppose A and B are regularized sets, we define the boolean operations:

- $A \cup^* B = \overline{\text{int}(A \cup B)}$
- $A \cap^* B = \overline{\text{int}(A \cap B)}$
- $A -^* B = \overline{\text{int}(A - B)}$
- $A^c * B = \overline{\text{int}(A^c)}$

III. CSG PRIMITIVES

A **constructive solid geometry** (CSG) model describes a polyhedron as regularized set operation on primitives which are the simplest solid objects used for the representation.

Primitives in CSG models are all semi-algebraic sets as follows:

1. **Half Space**: $H = \{(x, y, z) | a * x + b * y + c * z + d \geq 0\}$.
2. **Sphere**: $S = \{(x, y, z) | (x - x_0)^2 + (y - y_0)^2 + (z - z_0)^2 - r^2 \leq 0\}$.
3. **Cylinder**: $C = \{(x, y, z) | A = -x \sin(\theta) + y \cos(\theta) \cos(\phi) + z \cos(\theta) \sin(\phi), B = -y \sin(\phi) + z \cos(\phi), A^2 + B^2 \leq R^2\}$.
4. **Torus**: $T = \{(x, y, z) | (R - \sqrt{x^2 + y^2})^2 + z^2 \leq r^2\}$.

A polyhedron is limited by the intersection of half spaces.

IV. SUBDIVISION TREE

4.1 Box

B is a **box** if $B \subset \mathbb{R}^3$, $B = \{(x, y, z) | x_1 \leq x \leq x_2, y_1 \leq y \leq y_2, z_1 \leq z \leq z_2\}$, or simplified as $B = ([x_1, x_2], [y_1, y_2], [z_1, z_2])$, which is a regularized subset of $\mathbb{R}^3$.

4.2 Subdivision on box

Suppose $B = ([x_1, x_2], [y_1, y_2], [z_1, z_2])$, let $\bar{x} = \frac{x_1+x_2}{2}$, $\bar{y} = \frac{y_1+y_2}{2}$, $\bar{z} = \frac{z_1+z_2}{2}$. The subdivision on B is to subdivide B into eight subboxes by splitting at $x = \bar{x}$, $y = \bar{y}$ and $z = \bar{z}$.

4.3 Features

For a polyhedron $P \subset \mathbb{R}^3$, we use set V , E and F to represent its vertices, edges and faces respectively. The **features** $\Phi(P) = (V, E, F)$.

Then we define the features of a box B corresponding to polyhedron P as follows:

- $\Phi_0(B, P) = B \cap V$
- $\Phi_1(B, P) = \{e \mid e \in E, e \cap B \neq \emptyset\}$
- $\Phi_2(B, P) = \{f \mid f \in F, f \cap B \neq \emptyset\}$

$$\Phi(B, P) = (\Phi_0(B, P), \Phi_1(B, P), \Phi_2(B, P))$$

4.4 Elementary Box

We define a box B to be **elementary box** for a polyhedron P if one of the following conditions hold:

1. $B \cap P = \emptyset$.

Call B an **empty box** for P .

2. $B \subseteq P$.

Call B a **full box** for P .

3. Exists only one v st. $\Phi_0(B, P) = \{v\}$ and $\forall e \in \Phi_1(B, P)$ e is bounded by v and $\forall f \in \Phi_2(B, P)$ f is bounded by v .

Call B a **vertex box** for P .

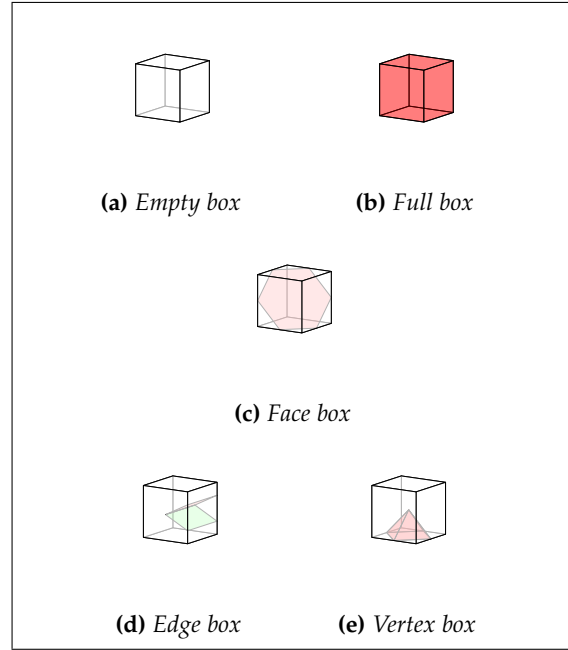


Figure 1: Elementary boxes

4. $V \cap B = \emptyset$, exists only one e st. $\Phi_1(B, P) = \{e\}$ and $\forall f \in \Phi_2(B, P)$ f is bounded by v .

Call B a **edge box** for P .

5. $V \cap B = \emptyset$, $E \cap B = \emptyset$, exists only one f st. $\Phi_2(B, P) = \{f\}$.

Call B a **face box** for P .

4.5 Definition

1. A **subdivision tree** is a tree where each interior node has degree 8 (Octree) and each node u is associated with a box $u.box$ st. if $u_0, \dots, u_7$ are the children of u . Then $\{u.Box : i = 0..7\}$ is the split of $u.box$.
2. An **adequate** subdivision tree of a box B_0 for a polyhedron P is a subdivision tree rooted at B_0 , in which all leaves are associated with elementary boxes for P .

4.6 Classification of elementary boxes

V. BOOLEAN OPERATIONS ON SUBDIVISION TREES

To compute the features of the union, intersection or difference of two polyhedra. We could compute the boolean operations on their subdivision trees.

5.1 Even up two trees

Give box B_0 and polyhedron P_1, P_2 . We have two adequate subdivision trees T_1 for P_1 , T_2 for P_2 .

Before we do the boolean operation on T_1, T_2 , it's necessary to make them congruent, which means $\forall$ node n_1 in T_1 , $\exists n_2$ in T_2 st. $n_1.\text{box} = n_2.\text{box}$, $\forall$ node n_2 in T_2 , $\exists n_1$ in T_1 st. $n_2.\text{box} = n_1.\text{box}$. We name this process **even up**.

The process of even up using BFS:

Input: T_1, T_2 are two adequate subdivision trees for P_1, P_2 respectively and both rooted at B_0 .

Output: The modified T_1 and T_2 which are compatible.

```

 $n_1 \leftarrow T_1.\text{root}.$ 
 $n_2 \leftarrow T_2.\text{root}.$ 
insert  $n_1$  into queue  $Q_1$ ,  $n_1$  into queue  $Q_2$ .
while  $Q_1 \neq \emptyset$  and  $Q_2 \neq \emptyset$  do:
   $n_1 \leftarrow Q_1.\text{dequeue}().$ 
   $n_2 \leftarrow Q_2.\text{dequeue}().$ 
  if ( $n_1.\text{box}$  is elementary for  $T_1$ 
    and  $n_2.\text{box}$  is elementary for  $T_2$ )
    then
      continue.
  if ( $n_1.\text{children} = \emptyset$ ) then
     $n_1.\text{children} \leftarrow n_1.\text{split}().$ 
     $T_1.\text{attach}(n_1.\text{children}, n_1).$ 
  elif ( $n_2.\text{children} = \emptyset$ ) then
     $n_2.\text{children} \leftarrow n_2.\text{split}().$ 
     $T_2.\text{attach}(n_2.\text{children}, n_2).$ 
   $Q_1.\text{enqueue}(n_1.\text{children}).$ 
   $Q_2.\text{enqueue}(n_2.\text{children}).$ 

```

Using this algorithm we should pay attention that we supposed the algorithm would stop. Otherwise we may never get the two trees congruent.

Lemma 1 Suppose B' is a subbox of B , P is a polyhedron, $\Phi(B', P) \subseteq \Phi(B, P)$.

proof. According to the definition of $\Phi(B, P)$:
 $\Phi(B, P) = (\Phi_0(B, P), \Phi_1(B, P), \Phi_2(B, P)).$

Since B' is a subbox of B , we also have:

$$\begin{cases} \Phi'_0(B, P) \subseteq \Phi_0(B, P) \\ \Phi'_1(B, P) \subseteq \Phi_1(B, P) \\ \Phi'_2(B, P) \subseteq \Phi_2(B, P) \end{cases}$$

Hence we get $\Phi(B', P) \subseteq \Phi(B, P)$.

Lemma 1 shows when a box gets smaller, the features has the trend to get less. We could extend it to

Lemma 2 Given a Box B and a polyhedron P , we get its adequate subdivision tree in limited subdivisions.

proof. We get its adequate subdivision tree, that is to say, we would get a state that all leaf subboxes of B are elementary for P .

We define $d(x, y)$ to be Euclidean distance between x and y (x, y can be vertex, edge or face). Given a non-elementary subbox B' having features $\Phi(B', P)$ for P , we discuss the following 6 simplest cases:

$\Phi(B', P)$ has (at least)

- two vertices v_1 and v_2 , $v_1 \neq v_2$. (1)
- two edges e_1 and e_2 , $e_1 \cap e_2 \cap B' = \emptyset$. (2)
- two faces f_1 and f_2 , $f_1 \cap f_2 \cap B' = \emptyset$. (3)
- a vertex v and an edge e , $v \notin e \cap B'$. (4)
- a vertex v and a face f , $v \notin f \cap B'$. (5)
- an edge e and a face f , $e \cap f \cap B' = \emptyset$. (6)

We define $l(B)$ is the side length of B . Then the diameter of B equals $\sqrt{3} * l(B)$.

For cases (1), (4) and (5), we simply keep subdividing B' . Then we choose the biggest box B'_m of its leaf subboxes. When the following conditions hold,

- $\sqrt{3} * l(B'_m) < d(v_1, v_2)$ (1)
- $\sqrt{3} * l(B'_m) < d(v, e)$ (4)
- $\sqrt{3} * l(B'_m) < d(v, f)$ (5)

We guarantee none of the leaf subboxes contains both features.

For cases (2), (3) and (6), the process is similar except we can't simply use the Euclidean distance between edges and faces since they may intersect outside the box B' . First we compute the line segments of edges and polygons of faces cut by B' , then similarly when

- $\sqrt{3} * l(B'_m) < d(e_1 \cap B', e_2 \cap B')$ (2)
- $\sqrt{3} * l(B'_m) < d(f_1 \cap B', f_2 \cap B')$ (3)
- $\sqrt{3} * l(B'_m) < d(e \cap B', f \cap B')$ (6)

We split the features into two different subboxes.

Therefore we have proved that during subdivision we would split all the unrelated features into different subboxes. *Lemma 2* holds.

Furthermore, we consider using the process above to compute $\Phi(B', P_1) \cup \Phi(B', P_2)$. We would get an adequate subdivision tree T_3 for both P_1 and P_2 . When we extend both T_1 and T_2 to T_3 , we make them compatible. We proved

Lemma 3 *Given a Box B , its adequate subdivision tree T_1 for polyhedron P_1 and adequate subdivision tree T_2 for polyhedron P_2 , we even T_1 and T_2 up in limited steps.*

Now we have two congruent adequate subdivision trees and then can step forward to the boolean operations.

5.2 Boolean operations

When we do boolean operations on two trees, we care about the leaves since they show the boundary condition. For simplification, we turn the boolean operations on trees to the same operations on two corresponding nodes (corresponding means two nodes are at the same position of their congruent trees). Suppose n_1, n_2 are two corresponding elementary leaf nodes of T_1, T_2 for P_1, P_2 respectively. Each of them represents one of the five types: empty box (EMB), full box (FUB), vertex box (VB), edge box (EDB), face box (FAB). We have the following rules (left operand is of n_1 , right operand is of n_2):

5.2.1 Empty boxes

- $EMB \cup B^* = B^*$
- $EMB \cap B^* = EMB$
- $EMB - B^* = EMB$
- $B^* - EMB = B^*$

5.2.2 Full boxes

- $FUB \cup B^* = FUB$
- $FUB \cap B^* = B^*$
- $\begin{cases} FUB - FUB = EMB \\ FUB - VB = VB \\ FUB - EDB = EDB \\ FUB - FAB = FAB \\ FUB - EMB = FUB \end{cases}$
- $B^* - FUB = EMB$

5.2.3 Vertex boxes

- $VB_1 \cup VB_2$
If VB_1 and VB_2 share the same vertex, the result is a vertex box. Otherwise the result is a non-elementary box which needs further subdivision.
- $VB \cup EDB$